
- Course Code: PA1458
- Name: Software Design

GenAI Policy

Note: Throughout this document, "you" refers to students enrolled in the course, "we" refers to examiners/supervisors/teaching team.

The goal of the course assignments is to demonstrate your ability to make use of basic software engineering tools that are an integral part of a modern software development environment. This includes your editor, compilers, debugger, automated test tools, programming language specific documentation tools, microservice container tools, etc.

All GenAI use must align with the objectives and intended learning outcomes defined in the course syllabus. GenAI should support, rather than replace, your demonstration of competencies such as critical thinking and scientific writing.

Sources

> GenAI Policy Template, Franz Zieris, Blekinge Insitute of Technology, 2026. Available at: https://github.com/bth-dipt-teaching/genai-policy/blob/main/template.md Licensed under CC BY 4.0.
>
> The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 2119.

The document further makes use of the AI Assessment Scale (AIAS) developed by Mike Perkins, Leon Furze, Jasper Roe, and Jason MacVaugh in 2023/2024.

General Principles

Full Responsibility and Accountability

- You **MUST** take total ownership of every word, figure, and line of code; verify all AI-assisted output against primary sources.
- You **MUST NOT** assume that the AI knows best (e.g., using "The AI said so" as a justification), that details do not matter, or that examiners will not notice.

- You **SHOULD** make sure you understand the output of your AI tool and then write/use your own interpretation of it.
- **Motivation**: You are effectively "signing" your assignments as your intellectual output. You must be able to explain, justify, and trace the origin of any part of it. In group work, accountability is collective; all authors are responsible for the entire content.
- **Example**: A student experimented with ChatGPT for generating some program code for a task, but then re-typed it by themselves on a level where they understand what the code does.

Do not Plagiarise

- **Comment**: This is a wider question than just the use of GenAI, but it applies equally to material produced by GenAI tools.
- You **MUST** properly cite and credit any material used in your assignments. This includes the *work* of others, and this includes output from GenAI tools.
- You **MUST NOT** copy-paste or otherwise present the work of others as if it was your own.
- You **SHOULD** make it a habit to always summarise text that you want to use and to immediately add a reference to where you got it from.
- You **MAY** include the prompts you used to generate code or text from a GenAI tool, in order to discuss it with your teacher.
- **Motivation 1**: The assignments are an opportunity for you to show what *you* have learned. By including sources and/or prompts we are able to help you learn more.
- **Motivation 2**: Plagiarism is a serious academic offence: You *must always* give credit to the original author(s).
- **Example**: A student asks Claude to generate different suggestions for the code for a particular task, but then writes their own version which is a combination of several suggestions. They add the prompts used in a source code comment and discuss these with the teacher.
- **Example**: A student finds several ready-made solutions to a programming task online, and writes their own solution inspired by what they have read. They include links to the used pages in a source code comment.

Use AI Tools to Explain, not to Do.

- You **MAY** use AI to explain different commands and overall workflow.
- You **MAY** see this as an opportunity to pratice your ability to find and read documentation and instruction manuals instead of relying on AI tools.
- You **SHOULD** make sure that you understand the tools that you use.
- You **SHOULD** verify what the AI tells you through independent tests.
- You **SHOULD NOT** rely on that the GenAI understand the tools for you.

- You **MUST NOT** blame the AI for any mistakes it does. Those mistakes are yours to catch.
- **Motivation**: You must at least be able to identify when GenAI tools suggests something wrong. Your value as an engineer is further increased if you are also able to work independently with minimal tool or AI support.
- **Example**: An engineer asked an AI tool to generate code to implement a feature and realised that the generated code would break the running system and cost millions of dollars. If they had not independently reviewed the generated code before committing they would not have caught this.
- **Comment**: In industry, the line between explaining and doing is blurred, but SHOULD only be crossed by engineers *who already know what they are doing*. You are still students and are still learning the trade.

Specific Guidelines

Installing Software

- You **MAY** use AI to assist with installing the required software.
- You **SHOULD** make sure you have a package manager installed that can automate most software installations, for the future.
- **Motivation**: This is not an examined part of the course and you should preferrably not get stuck on this point so you are free to do as you please. In order to gain more control and understanding, it is better if you use a dedicated package manager.
- **Example**: A student is struggling to get git working in WSL2, so they ask Copilot for help, and follows the instructions received.

Creating UML diagrams and Use Cases

- You **MUST** understand the fundamental object oriented design principles that form the basis of each UML diagram.
- You **MUST** understand how to use what is produced in one UML diagram as a starting point for creating the next diagram.
- You **MAY** use AI tools to help you with the *syntax* of a diagram.
- You **MAY** use AI tools to explain how to improve a software design using fundamental object oriented design principles.
- You **MUST NOT** use AI tools to generate the entire diagram for you.
- **Motivation**: Understanding and applying fundamental object oriented design principles is at the core of this course, and is a critical skill as a software engineer.
- **Example**: A student struggles to identify the relevant input for creating a particular UML diagram, and asks Claude to summarise the previous design step. The student then creates the required UML diagram on their own.
- **Example**: Having manually created a UML diagram, a student asks

ChatGPT whether it correctly follows UML syntax. Where the AI tool identify issues, the student verifies the correct syntax against the course book.

Refactoring Class Diagrams

- You **MAY** use AI tools to explain what is expected of you.
- You **MAY** use AI tools to explain and give examples on how a specific design pattern work.
- You **MUST NOT** use AI tools to generate the updated design.
- **Motivation**: The assignment makes use of different skills, i.e. recognising a design pattern already in place, recognising which design pattern to use for a given problem, and being able to apply a specific design pattern in a particular context. All three are critical skills for becoming a good software engineer.
- **Example**: A student uses AI for getting ideas on how to apply design patterns in an existing design, but then modifies the design manually based on their own understanding.

Implementation

- You **MUST NOT** use GenAI tools to write any source code or code documentation.
- You **SHOULD** discuss with your friends.
- **Motivation**: The point of this assignment is to understand the knowledge gap between a software design and the resulting software implementation, and to start exercising the skills necessary to bridge this gap. Using an AI tool for this would seriously impede this understanding. Discussing with your friends is a vital learning tool, and the big difference between that and using an AI tool is that your friends know when to stop explaining and let you find the solution on your own.
- **Example**: A student is struggling with how to write the source code for an assignment, so they discuss with their classmates who show and explain how they solved the same task.
- **Example**: A student used a GenAI tool to generate source code from the given software design, and as a result fails to learn how to create a design that is easier to implement.

Unit Testing

- You **MAY** use GenAI tools to write the automated tests.
- You **MUST NOT** use GenAI to give examples of which unit tests to write.
- **Motivation**: The learning outcome of this assignment is to understand how software design artefacts can be tested, and to further practice the idea that unit tests should go hand in hand with any software development

activity. There is a risk, if using an AI to generate the tests, that you simply do not look closely enough at the software design to understand what makes it testable or not testable.

- **Example**: A student studies the use cases and the software design and decides which unit tests should be created, and then instructs their AI coding agent to generate these tests. They then review the generated tests to ensure that they actually test the right functionality in the right way.